main.py

Visualize

Run

```python
import heapq

graph = {
    'S': {'A': 1, 'B': 4},
    'A': {'B': 2, 'C': 5, 'D': 12},
    'B': {'C': 2},
    'C': {'D': 3, 'G': 7},
    'D': {'G': 2},
    'G': {},
}
h = {'S': 7, 'A': 6, 'B': 4, 'C': 4, 'D': 2, 'G': 0}

def a_star(start, goal):
    open_list = [(h[start], 0, start, [start])]
    best_g = {start: 0}
    while open_list:
        f, g, node, path = heapq.heappop(open_list)
        if node == goal:
            return path, g
        for nb, cost in graph[node].items():
            ng = g + cost
            if ng < best_g.get(nb, float('inf')):
                best_g[nb] = ng
                heapq.heappush(open_list, (ng + h[nb], ng, nb, path + [nb]))
    return None, float('inf')
```

Output

```
Path found: S -> A -> B -> C -> D -> G
Total cost: 10
```

main.py

Visualize

Run ▶

```
    'A': {'B': 2, 'C': 5, 'D': 12},
    'B': {'C': 2},
    'C': {'D': 3, 'G': 7},
    'D': {'G': 2},
    'G': {},
}
h = {'S': 7, 'A': 6, 'B': 4, 'C': 4, 'D': 2, 'G': 0}

def a_star(start, goal):
    open_list = [(h[start], 0, start, [start])]
    best_g = {start: 0}
    while open_list:
        f, g, node, path = heapq.heappop(open_list)
        if node == goal:
            return path, g
        for nb, cost in graph[node].items():
            ng = g + cost
            if ng < best_g.get(nb, float('inf')):
                best_g[nb] = ng
                heapq.heappush(open_list, (ng + h[nb], ng, nb, path + [nb]))
    return None, float('inf')

path, cost = a_star('S', 'G')
print("Path found:", ' -> '.join(path))
print("Total cost:", cost)
```

Output

```
Path found: S -> A -> B -> C -> D -> G
Total cost: 10
```